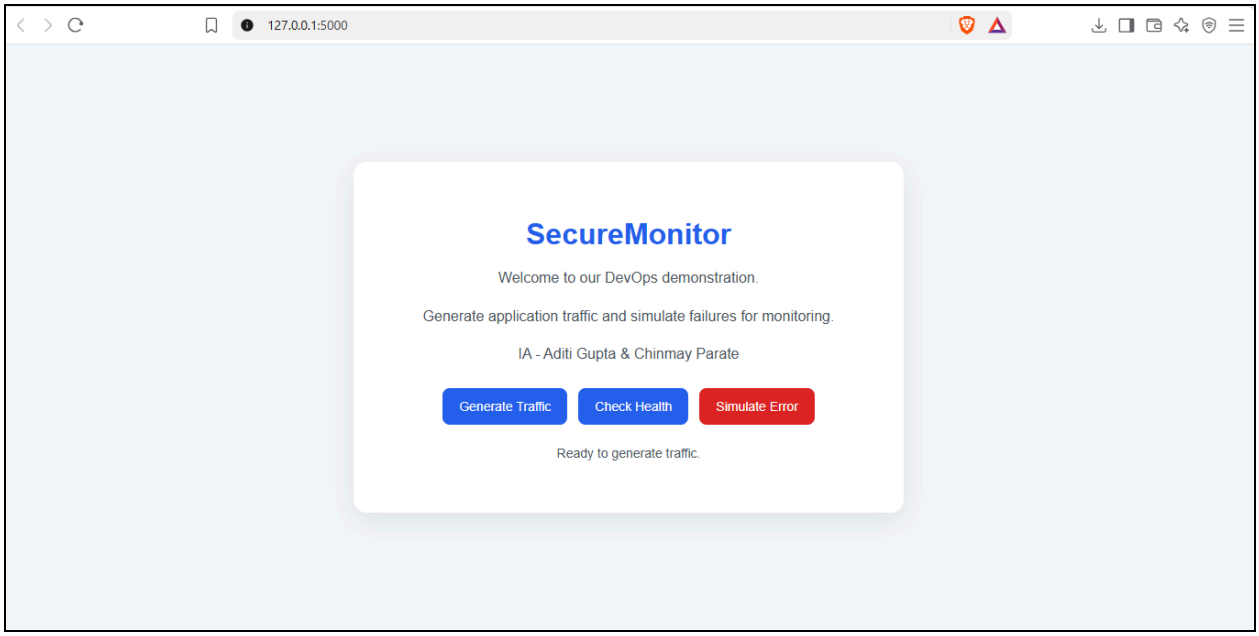


Status check

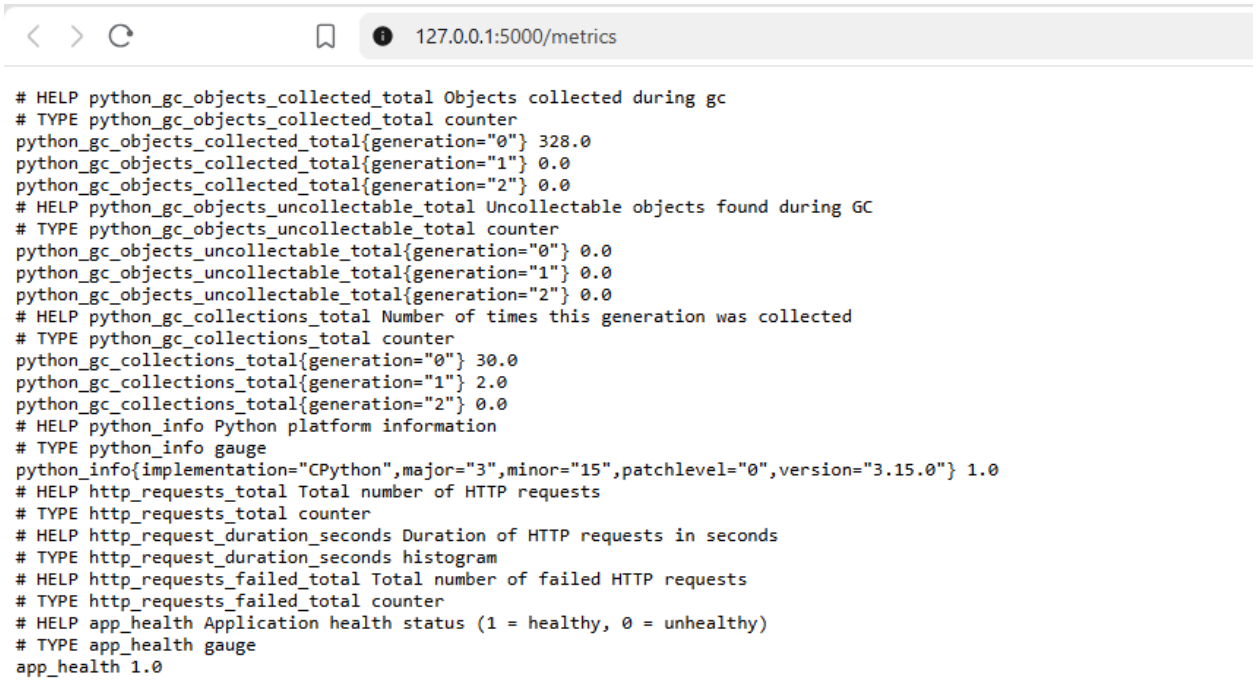
```
(.venv) PS C:\Users\SERVER\Desktop\DevOps_IA_18_20> Invoke-RestMethod http://127.0.0.1:5000/health

status
------
healthy


(.venv) PS C:\Users\SERVER\Desktop\DevOps_IA_18_20> Invoke-RestMethod http://127.0.0.1:5000/api/data

data       message
----       -------
@{value=3} SecureMonitor API is running
```

Your `/metrics` page is responding correctly, and all four Phase 2 metrics are present.



Verification results

Metric	Result	Meaning
`http_requests_total`	✅ Present	Counts HTTP requests
`http_request_duration_seconds`	✅ Present	Measures request duration
`http_requests_failed_total`	✅ Present	Counts failed HTTP requests

`app_health`	✅ Present — `1.0`	Application reports healthy

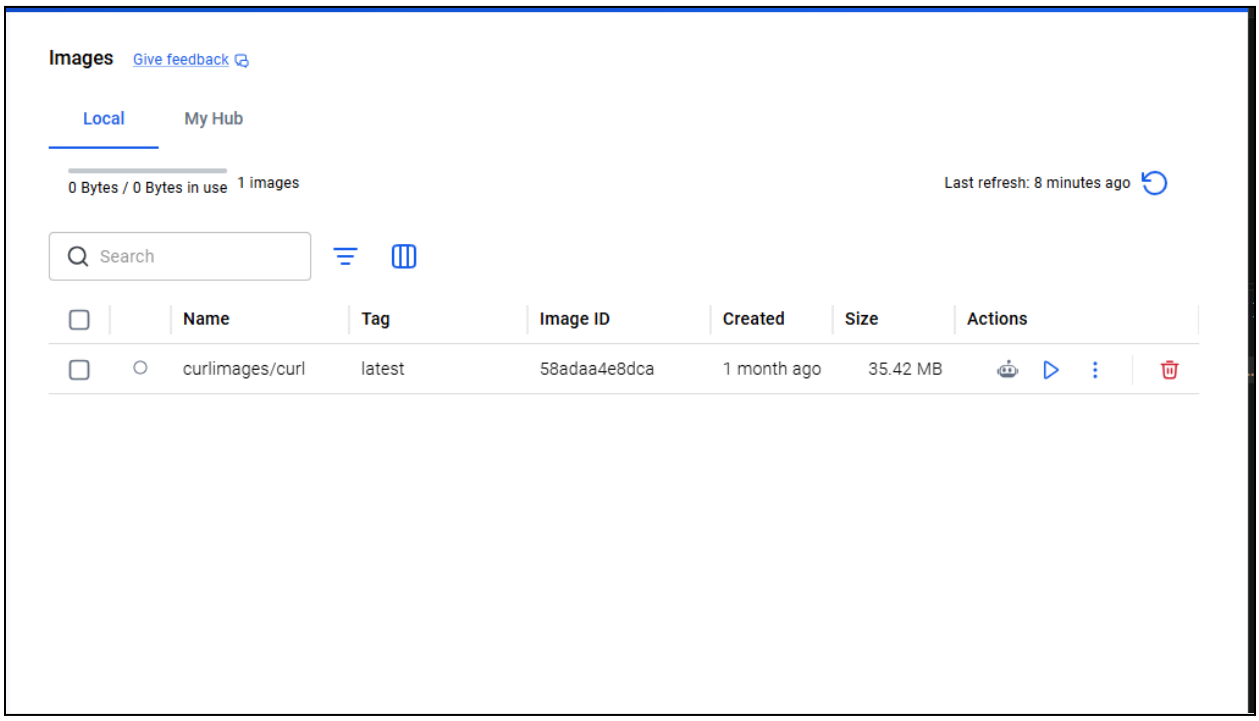
Metric	Your result	Status
`http_requests_total`	Request counter implemented	✅
`http_request_duration_seconds`	`/api/data`: 4 requests; `/api/error`: 1 request	✅
`http_requests_failed_total`	`/api/error` with status `500`: count `1.0`	✅
`app_health`	`1.0` — healthy	✅

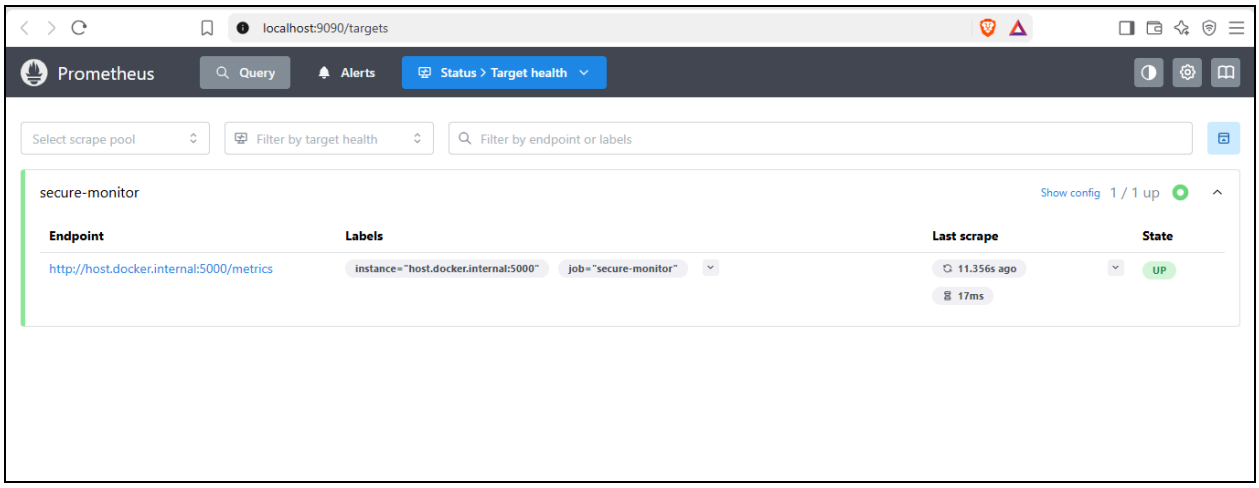
Your duration histogram also records the request count and total elapsed time. For example, `/api/data` has a duration sum of approximately `0.0004811` seconds across four requests.



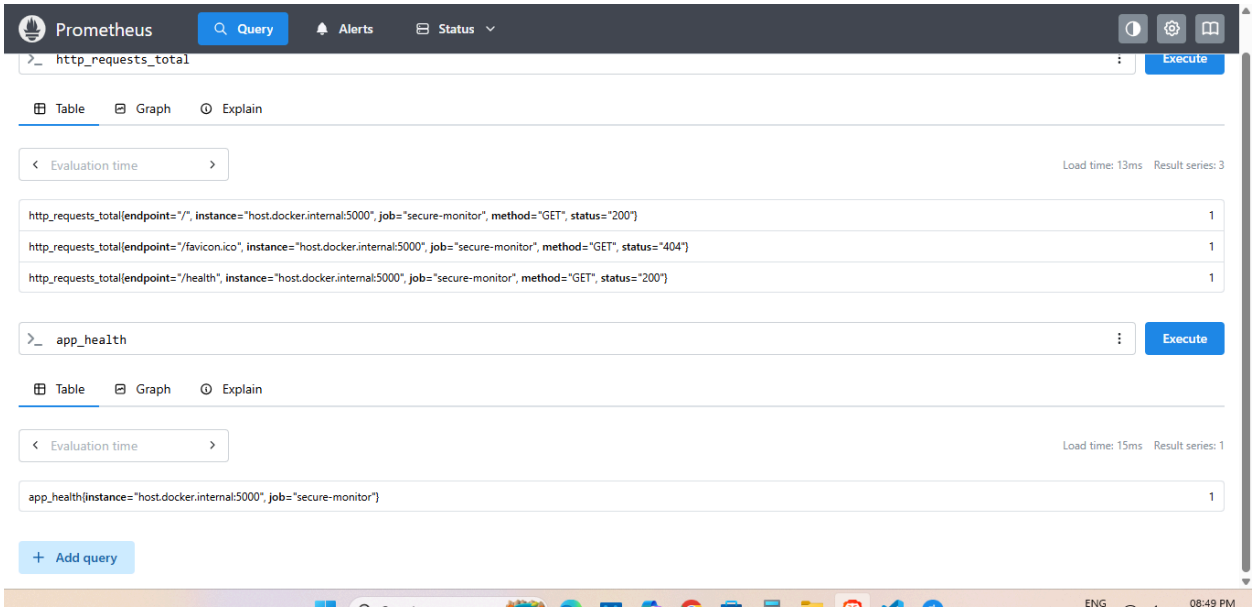
Docker connection verified

```
(.venv) PS C:\Users\SERVER\Desktop\DevOps_IA_18_20> & "$env:USERPROFILE\AppData\Local\Programs\DockerDesktop\resources\bin
\docker.exe" run --rm curlimages/curl:latest -sS http://host.docker.internal:5000/health
Unable to find image 'curlimages/curl:latest' locally
latest: Pulling from curlimages/curl
b18ba029a163: Pull complete
Digest: sha256:58adaa4e8dca9c988bae2aba4ab3434a0bb2da16bbe3f92dec39ec7785166777
Status: Downloaded newer image for curlimages/curl:latest
{"status":"healthy"}
```

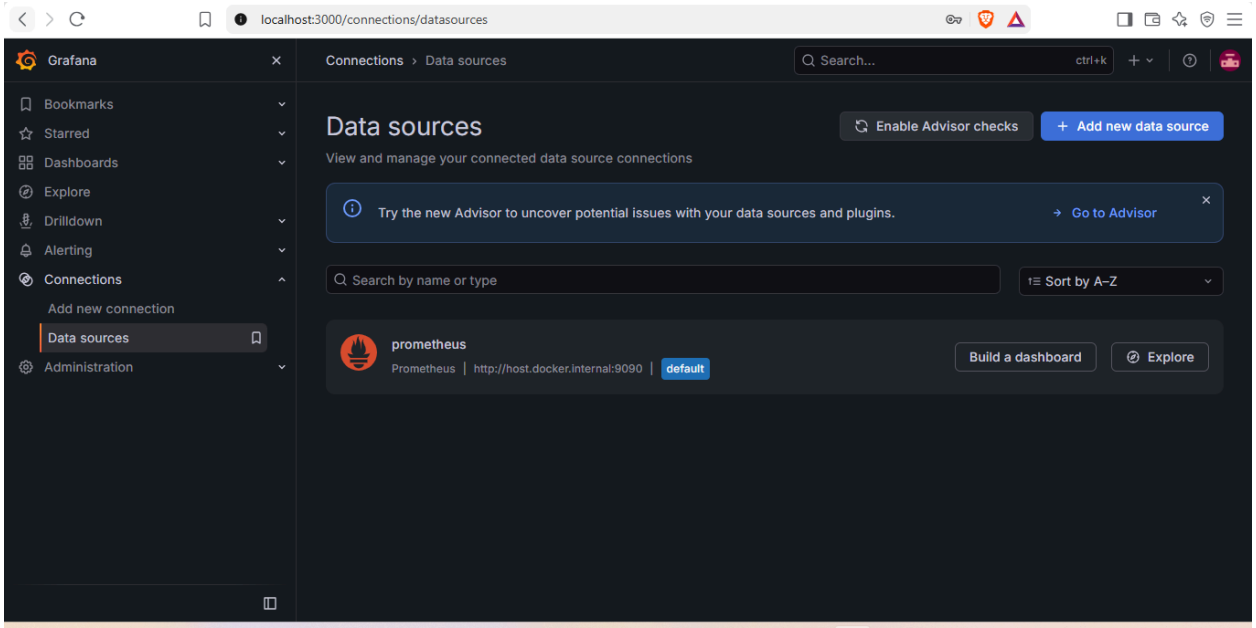




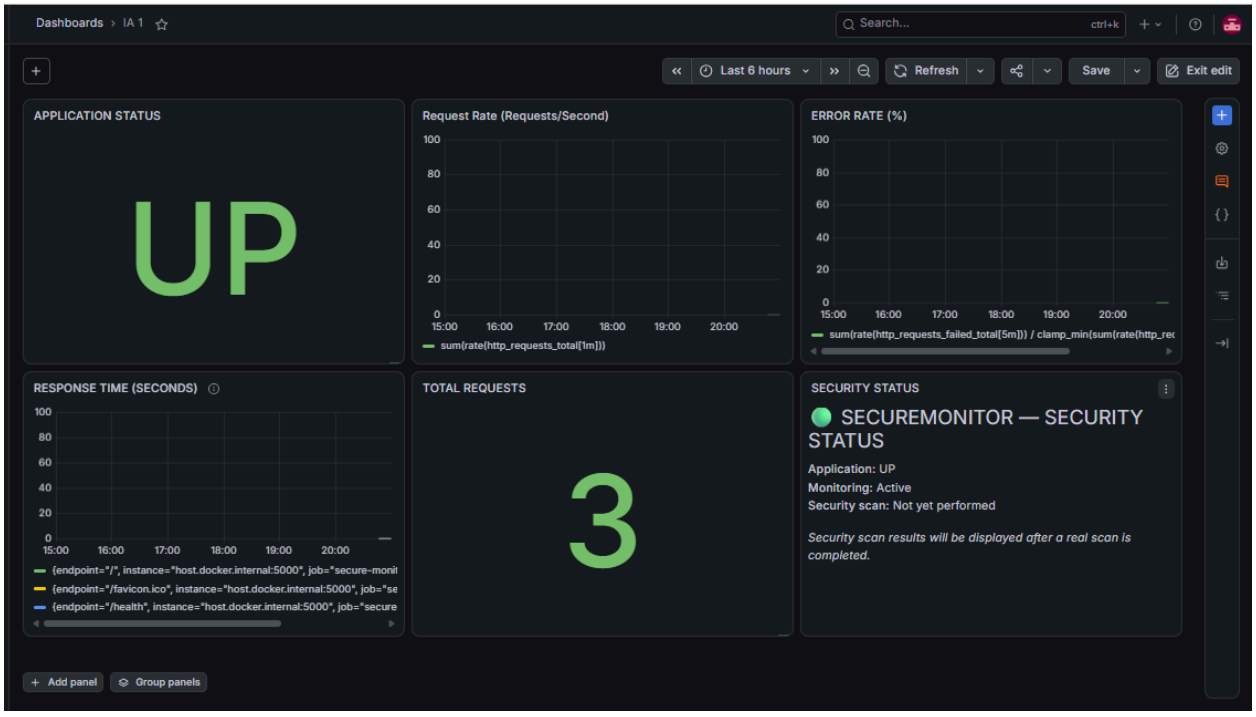
Prometheus Target Health Verification — The Prometheus dashboard confirms that the `secure-monitor` target is successfully connected to the Flask application. The `/metrics` endpoint is being scraped successfully, and the target status is **UP (1/1)**, confirming successful integration between Flask and Prometheus.



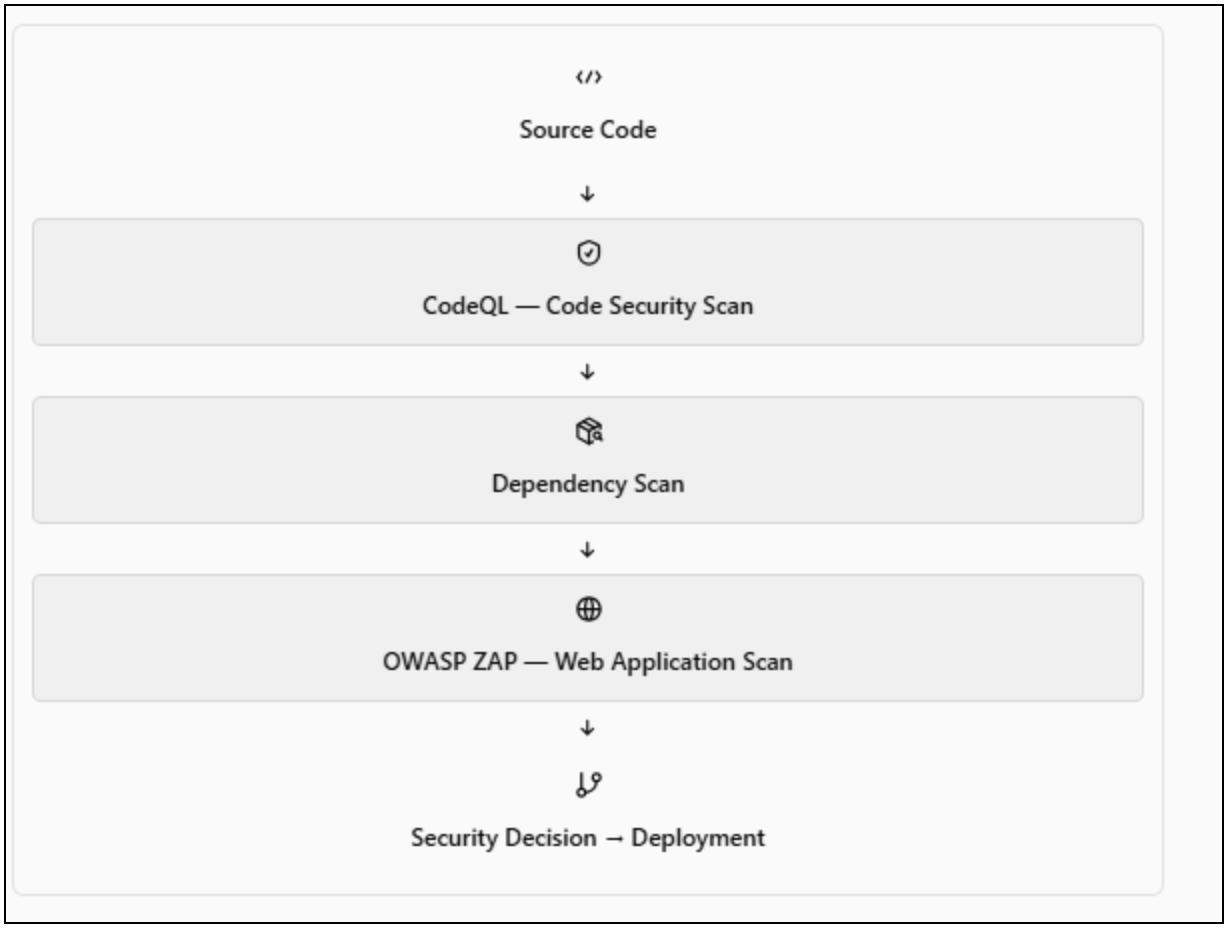
Prometheus Metrics Query Verification — The Prometheus query interface successfully displays the `http_requests_total` and `app_health` metrics collected from the Flask application. The `http_requests_total` metric shows three request series for the `/`, `/favicon.ico`, and `/health` endpoints, while `app_health` returns a value of `1`, indicating that the application is healthy.



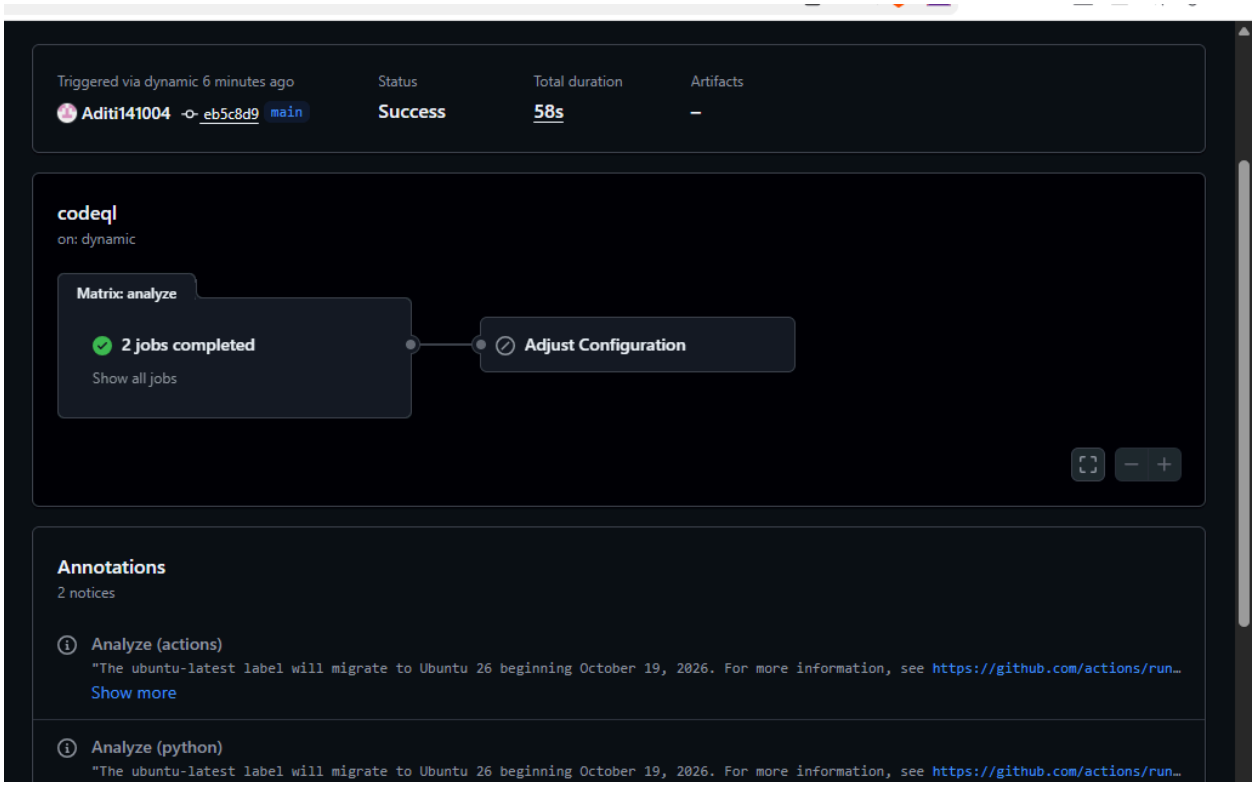
Prometheus added as a data source on Grafana



SecureMonitor Grafana Monitoring Dashboard — The Grafana dashboard integrates with Prometheus to visualize Flask application health, request rate, error rate, response time, and total request count. The Security Status panel summarizes the current monitoring state and indicates that a security scan has not yet been performed. The dashboard provides a centralized view of application performance and operational health.



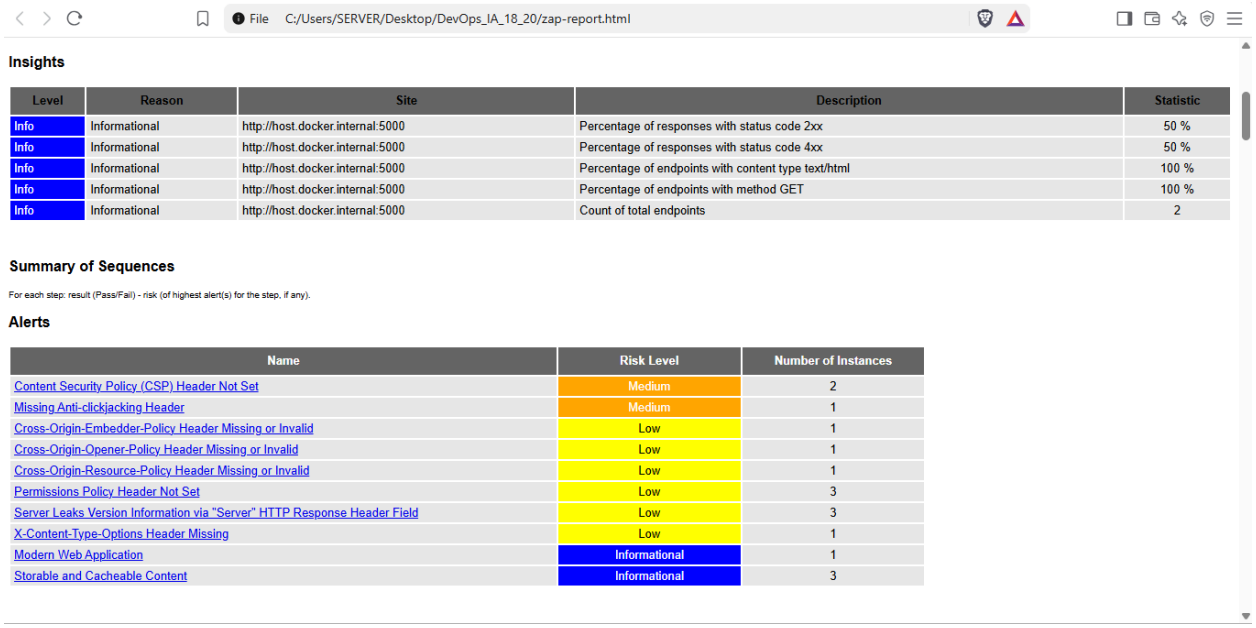
DevSecOps pipeline



ZAP scan result

Completed

0	8
FAIL alerts	Warning types
59	0
PASS checks	Alerts in progress



File C:/Users/SERVER/Desktop/DevOps_IA_18_20/zap-report.html

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://host.docker.internal:5000	Percentage of responses with status code 2xx	50 %
Info	Informational	http://host.docker.internal:5000	Percentage of responses with status code 4xx	50 %
Info	Informational	http://host.docker.internal:5000	Percentage of endpoints with content type text/html	100 %
Info	Informational	http://host.docker.internal:5000	Percentage of endpoints with method GET	100 %
Info	Informational	http://host.docker.internal:5000	Count of total endpoints	2

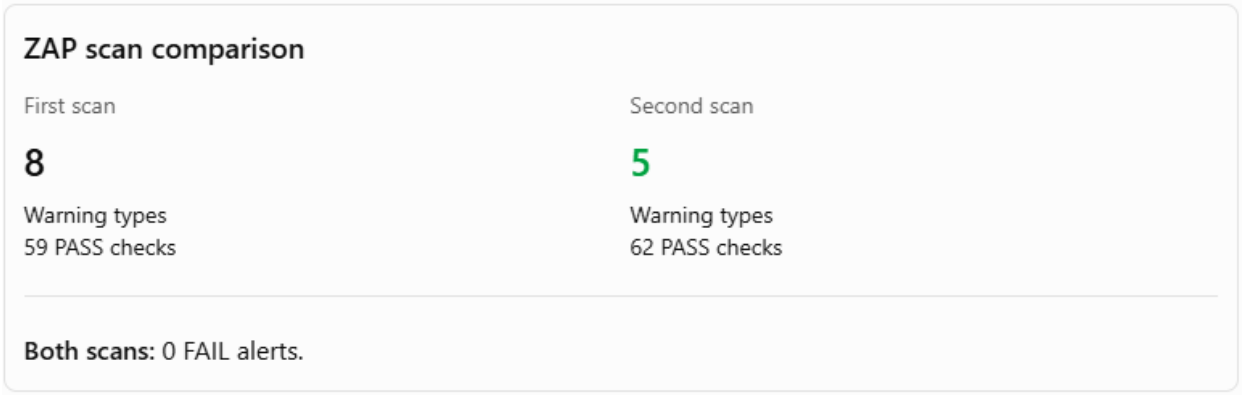
Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	2
Missing Anti-clickjacking Header	Medium	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	1
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	1
Permissions Policy Header Not Set	Low	3
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	3
X-Content-Type-Options Header Missing	Low	1
Modern Web Application	Informational	1
Storable and Cacheable Content	Informational	3

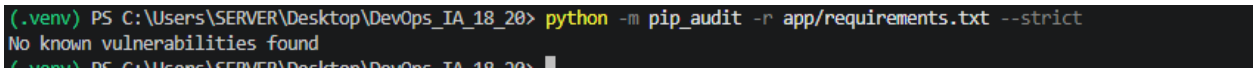
Made improvements in the code after seeing this result



ZAP scan comparison

First scan	Second scan
8	5
Warning types	Warning types
59 PASS checks	62 PASS checks

Both scans: 0 FAIL alerts.



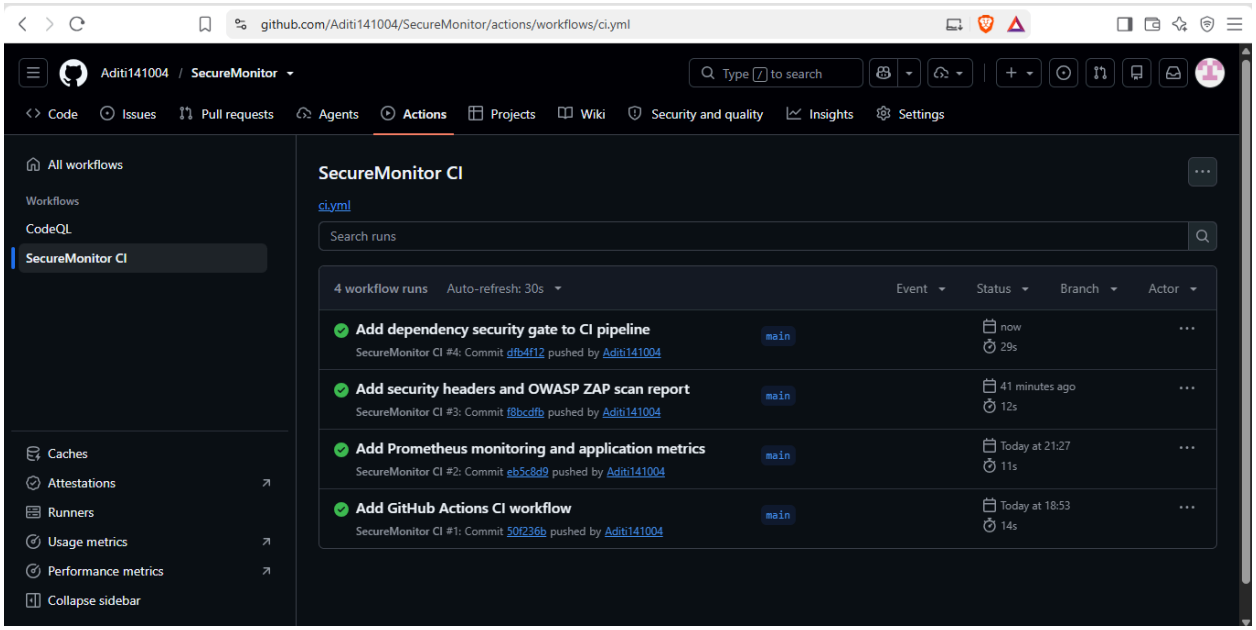
```
(.venv) PS C:\Users\SERVER\Desktop\DevOps_IA_18_20> python -m pip_audit -r app/requirements.txt --strict
No known vulnerabilities found
```

Automated tests also passed

```
(.venv) PS C:\Users\SERVER\Desktop\DevOps_IA_18_20> python -m pytest tests/ -v
============================================= test session starts =============================================
platform win32 -- Python 3.15.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\SERVER\Desktop\DevOps_IA_18_20\.venv\Scripts\pytho
n.exe
cachedir: .pytest_cache
rootdir: C:\Users\SERVER\Desktop\DevOps_IA_18_20
plugins: platformdirs-4.13.0
collected 5 items

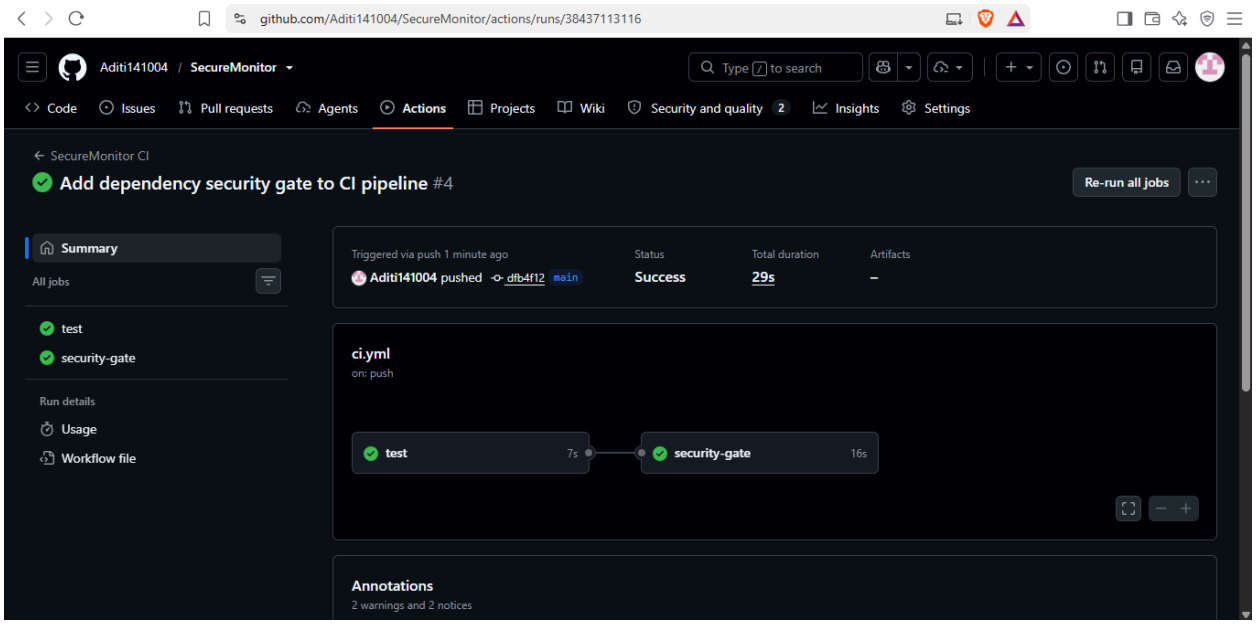
tests/test_app.py::test_home PASSED                                                                  [ 20%]
tests/test_app.py::test_health PASSED                                                                [ 40%]
tests/test_app.py::test_api_data PASSED                                                              [ 60%]
tests/test_app.py::test_api_error PASSED                                                             [ 80%]
tests/test_app.py::test_metrics PASSED                                                               [100%]

============================================== 5 passed in 0.36s ==============================================
(.venv) PS C:\Users\SERVER\Desktop\DevOps_IA_18_20>
```



SecureMonitor CI Pipeline Execution on GitHub Actions

The GitHub Actions dashboard displays successful execution of the SecureMonitor CI workflow following the addition of the dependency security gate. The latest workflow run, associated with commit `dfb4f12`, completed successfully. Previous runs for the CI workflow, Prometheus monitoring integration, and OWASP ZAP security report also show successful completion.



Successful Execution of the SecureMonitor DevSecOps Security Gate

The GitHub Actions workflow completed successfully for commit `dfb4f12`. Both the automated testing job (`test`) and the dependency security audit job (`security-gate`) passed. The security gate checks Python dependencies for known vulnerabilities using `pip-audit` and prevents the workflow from succeeding if the audit fails.

Grafana Dashboard after running the tests, making changes to the repo:

